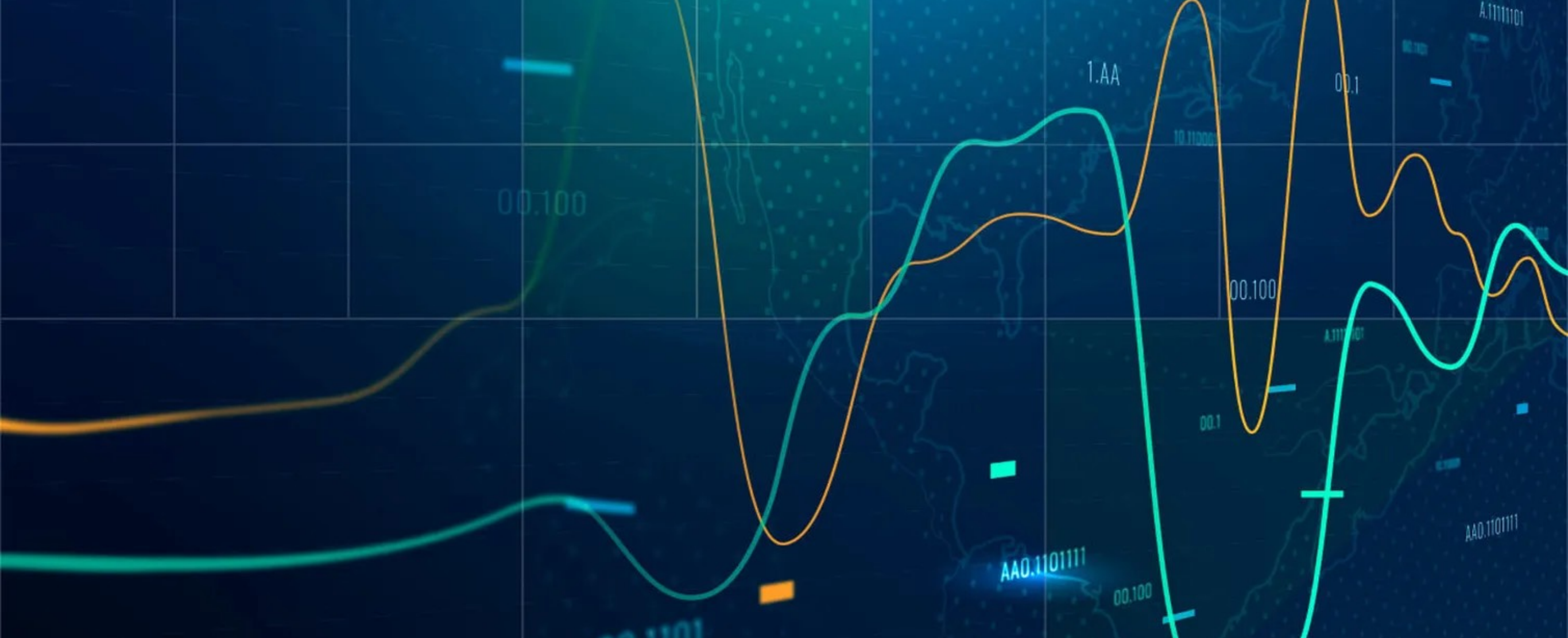


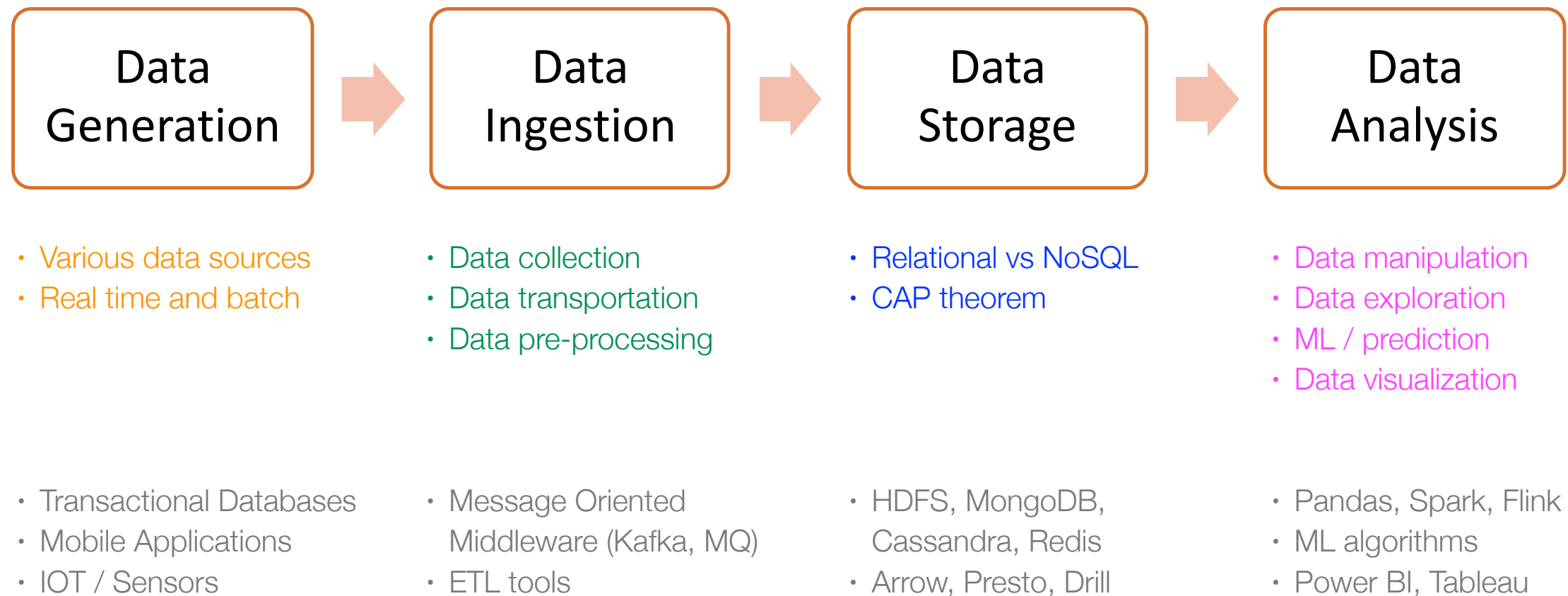


2110446 - Data Science and Data Engineering

Timeseries Datastore - Prometheus

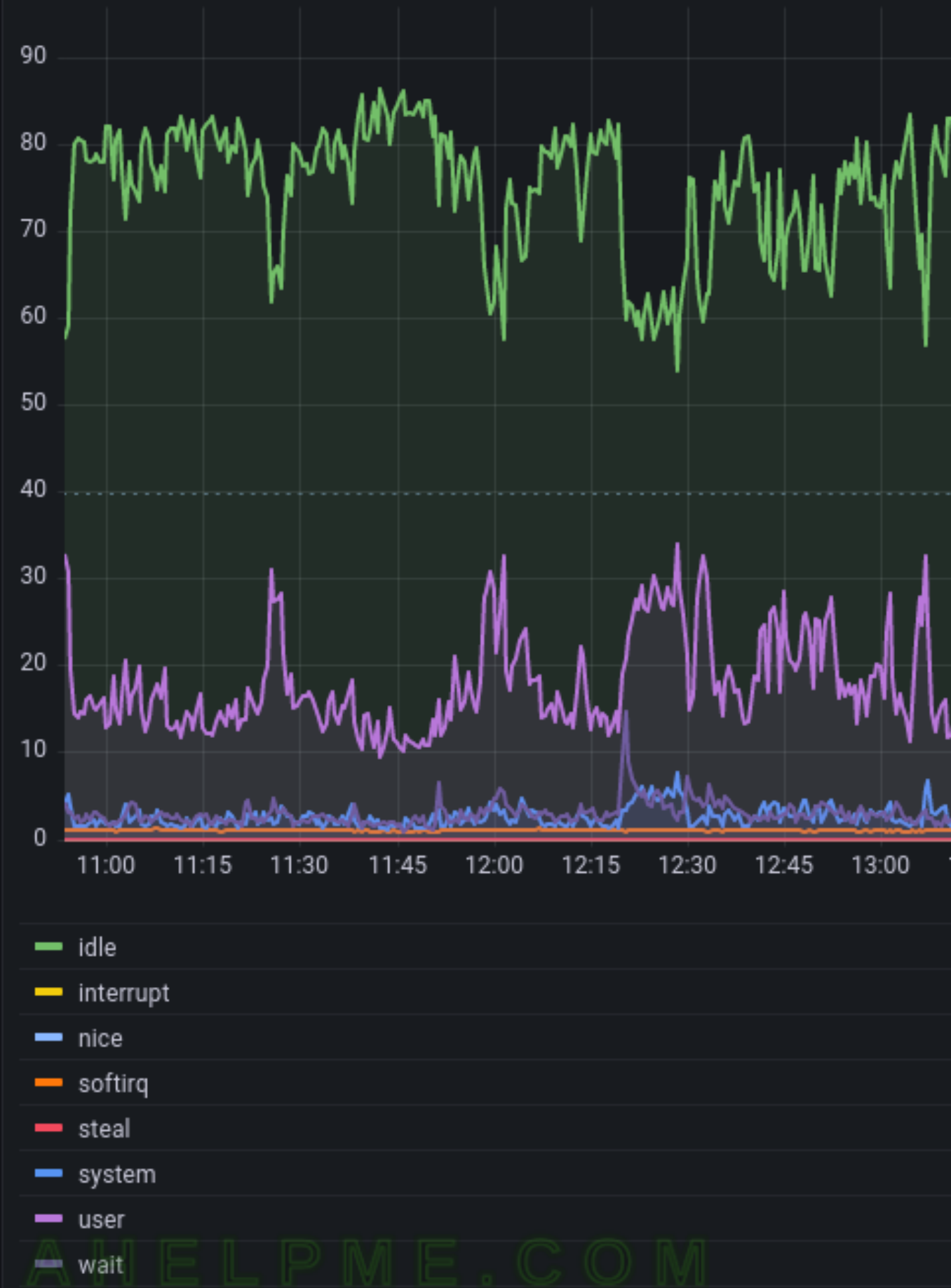
Asst.Prof. Natawut Nupairoj, Ph.D.
Department of Computer Engineering
Chulalongkorn University
natawut.n@chula.ac.th

Data Lifecycle

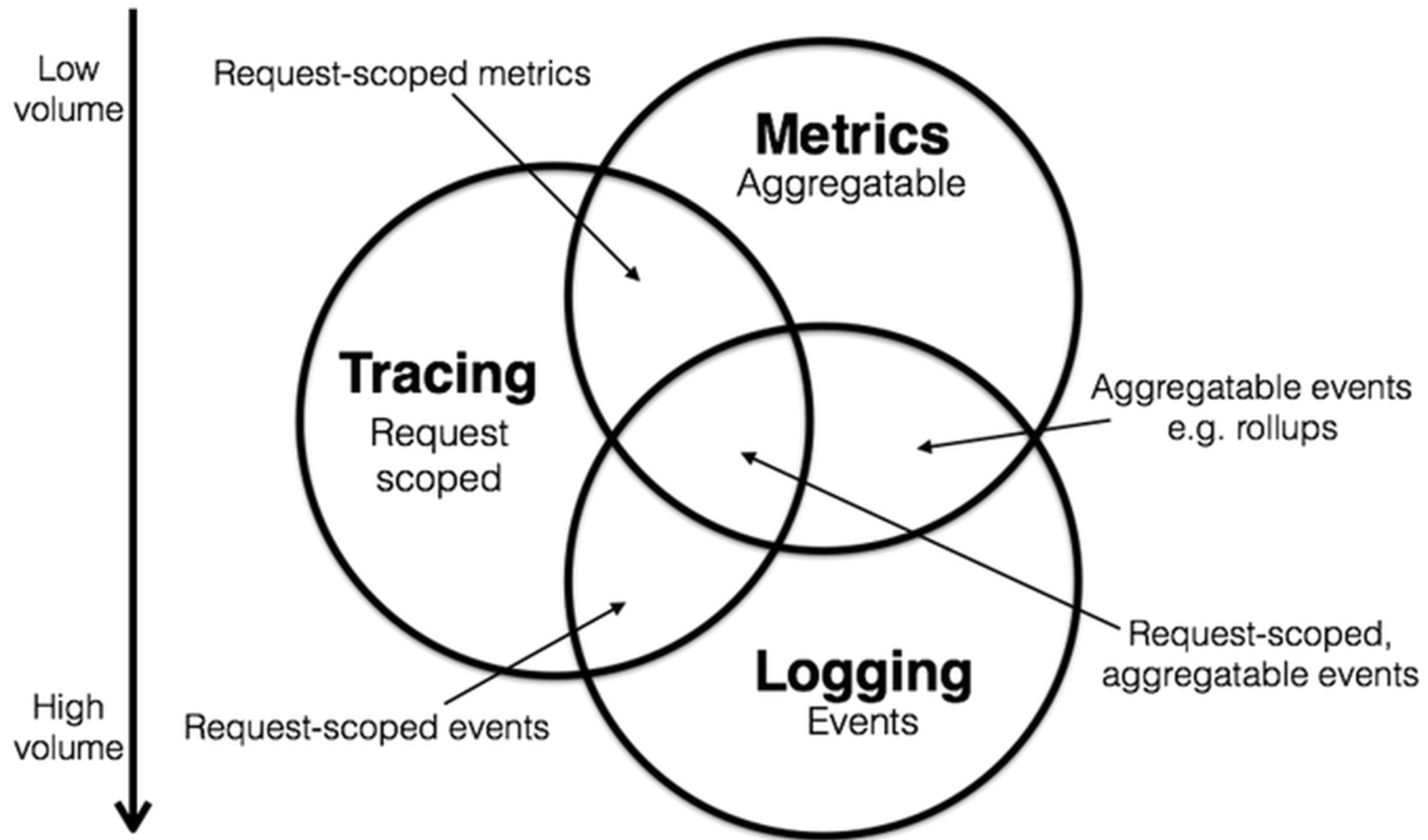


TSDb: Time Series Database

- A sequence of data points recorded at specific time intervals
- The Timestamp is the primary key and the most critical dimension
- Example:
 - CPU workloads from servers
 - Heart rate monitor from ICU
 - Stock market quotes
 - Air quality sensors
- **Time series datastore is the heart and soul of modern observability**



Three Pillars of Observability

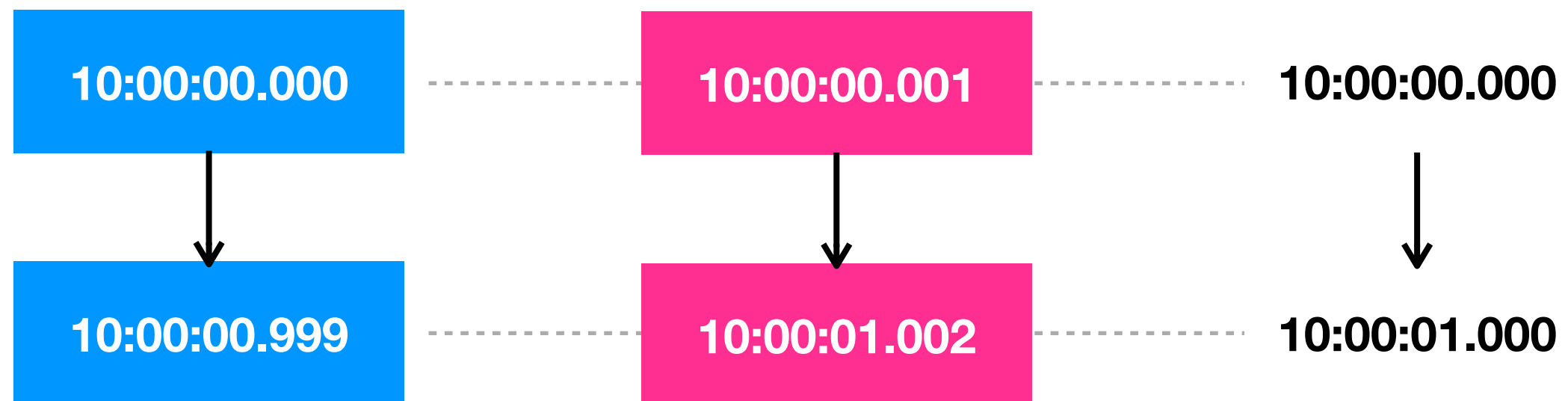


Time Series Workload Characteristics

- **High Ingestion**: Constant streams of appends
- **Immutable**: Historical data cannot be changed
- **Temporal Locality**: Recent data is queried 95% of the time
- **High Compression Ratio**: Several techniques are designed for time series data e.g. delta, delta-of-delta, etc.
- **Downsampling for Long-Term Retention**: reduces storage footprint by aggregating high-frequency data into larger time buckets (e.g., seconds to hours)

NoSQL on Time Series Data Limitations

- **Index Bloat:** Indexing timestamps in a Document DB causes index sizes to explode, often exceeding the actual data size
- **Retention Pain:** Deleting billions of documents in a NoSQL cluster creates massive I/O overhead and table locks
- **Alignment:** TSDBs handle "sample alignment" (moving data into consistent time buckets e.g. every 15-min) natively; NoSQL requires app logic



Data Stores and Time Series Data

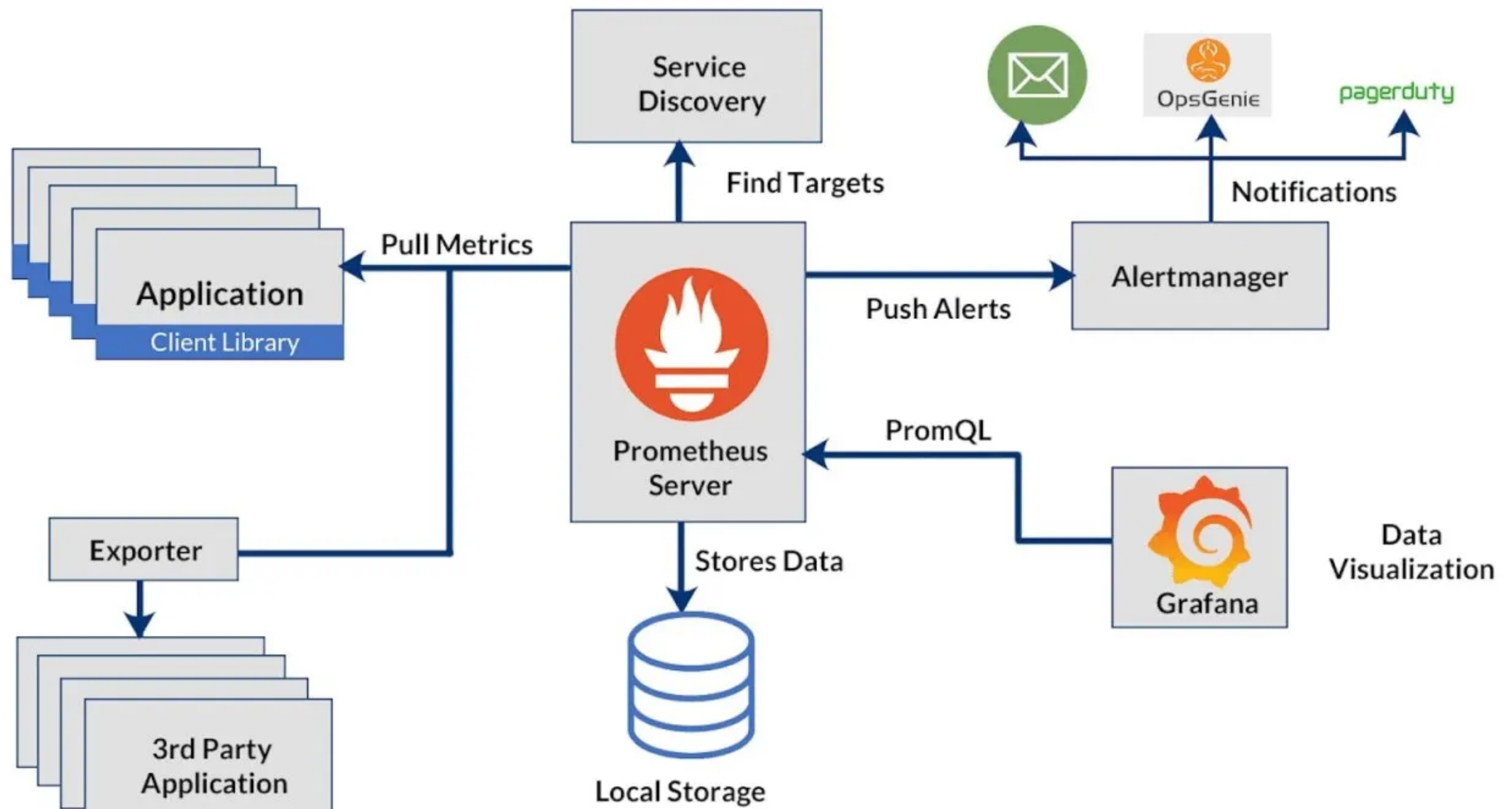
Capability	Relational (RDBMS)	NoSQL (Document)	TSDB (Prometheus/Influx)
Ingestion	Transaction-Heavy	Scalable but Generic	Optimized Appends
Storage Footprint Reduction	General (Inefficient)	Minimal	Delta-of-Delta (90% Red.) Downsampling
Aggregation	Complex Joins	Map-Reduce / Agg Pipes	Native Temporal Ops

TSDB with Prometheus

- An open-source systems monitoring and alerting toolkit
- Pulls (scrape) metrics from a client (target) over http
- Places the data into its time series database
- Query using its own DSL (PromQL)
- Have its own Web UI and integrate seamlessly with Grafana



Prometheus Architecture



Running Prometheus

- For this tutorial, we will run 3 containers:
 - prometheus (port 9090) - main TSDB with Web UI (via 9090)
 - node-exporter (port 9091) - an agent collecting hw/os metrics
 - grafana (port 3000) - visualization platform
- See “docker-compose.yml” and “prometheus/prometheus.yml” for more details

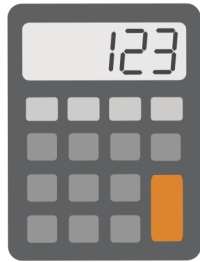
PromQL

- Prometheus Query Language (PromQL) is a powerful and flexible language used to query and manipulate time-series metrics in Prometheus
- Key Concepts
 - Metrics and their types
 - Data types: instant vector, range vector, scalar, string
 - Expression: operators, functions

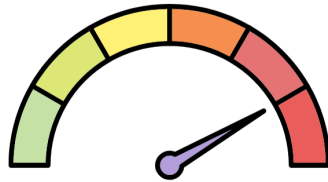
Metrics

- Numeric representation of data measured over intervals of time
- Can be used to represent the behavior of a system over intervals of time in the present and future
- Anatomy of a metric
 - Metric Name: The "what" (e.g., node_cpu_seconds_total)
 - Labels: Dimensions for filtering (e.g., region="asia-southeast-1a", instance="server-1")
 - Timestamp: Unix epoch in milliseconds
 - Value: Standardized 64-bit floating point

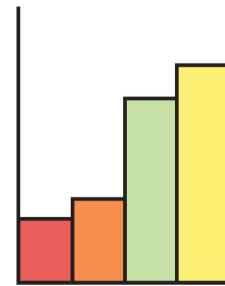
Prometheus Metric Types



Counter



Gauge

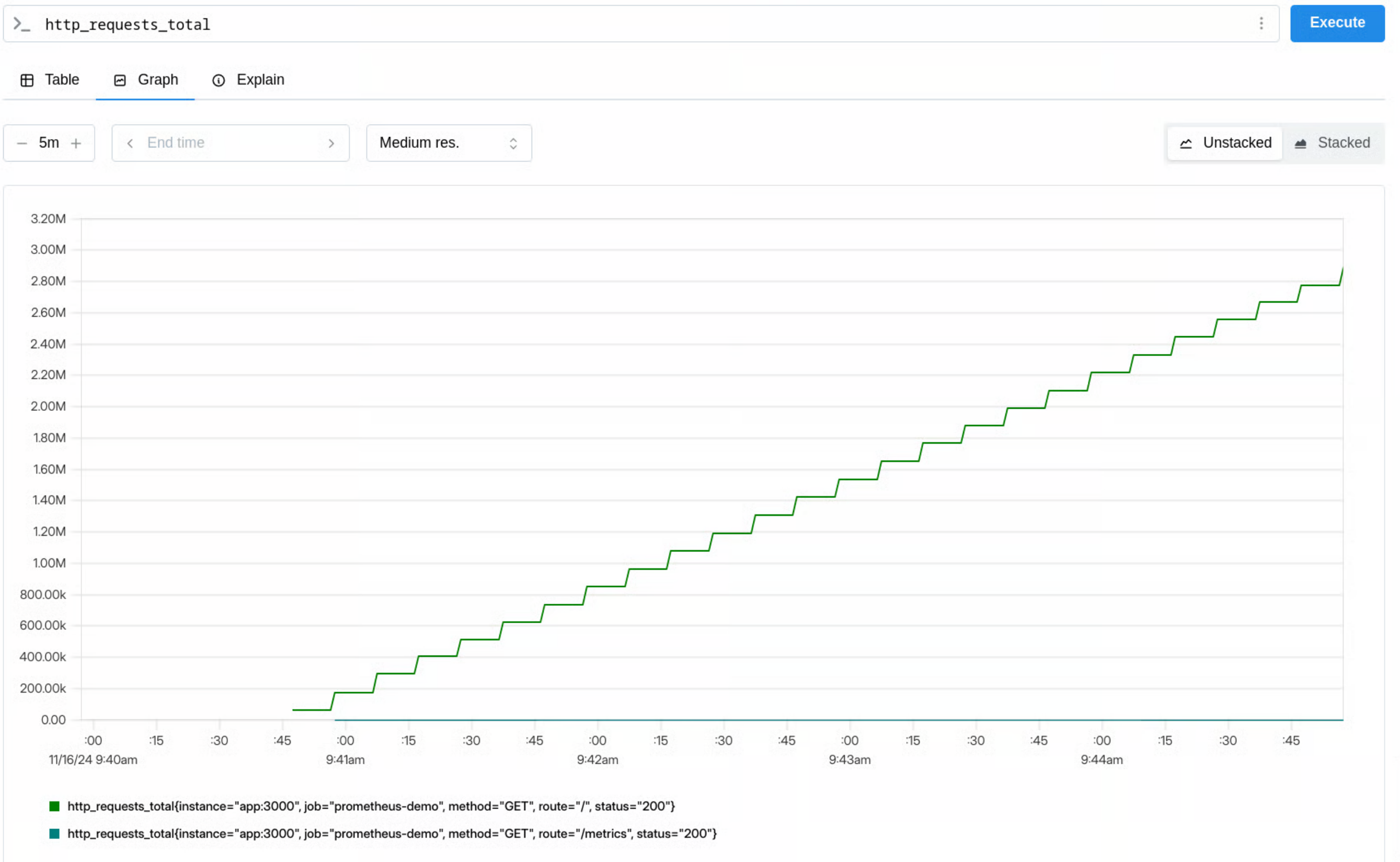


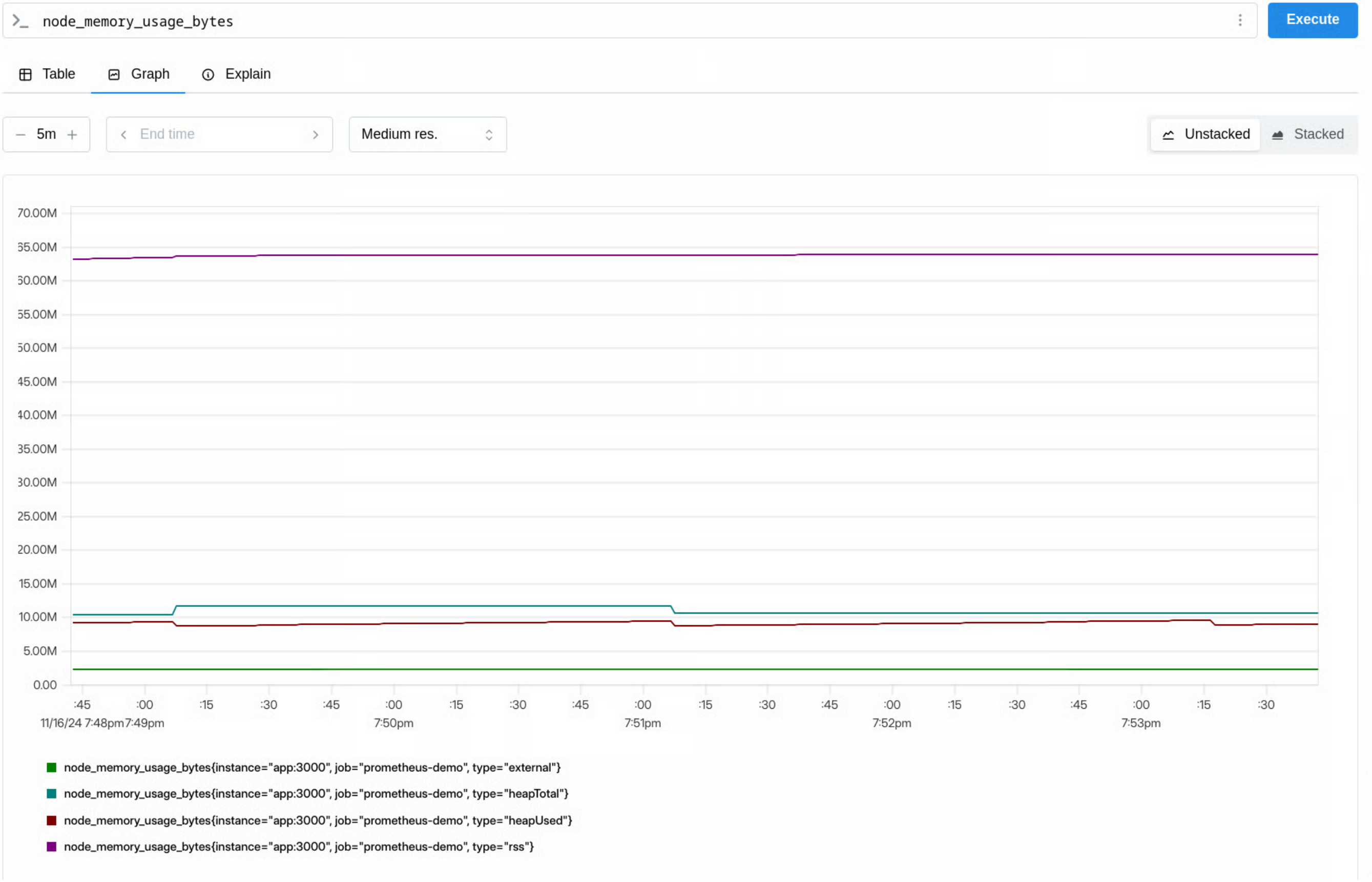
Histogram



Summary

- **Counters** for tracking ever-increasing values, like the total number of exceptions thrown
- **Gauges** for measuring fluctuating values, such as current CPU usage
- **Histograms** for observing the distribution of values within predefined buckets (calculation on the server)
- **Summaries** for calculating quantiles (percentiles) of observed values (calculation on the client, sliding-window)





>_ http_request_duration_seconds_bucket

Execute

Table

Graph

Explain

< Evaluation time >

Load time: 9ms Result series: 12

http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.005"}	176
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.01"}	176
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.025"}	176
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.05"}	176
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.1"}	177
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.25"}	2335
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="0.5"}	5751
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="1.0"}	5754
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="2.5"}	5754
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="5.0"}	5754
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="10.0"}	5754
http_request_duration_seconds_bucket{instance="app:3000", job="prometheus-demo", le="+Inf"}	5754

+ Add query

Data Types

- In PromQL, an expression or sub-expression can evaluate to one of four types:
 - Instant vector - a set of time series containing a single sample for each time series, all sharing the same timestamp
 - Range vector - a set of time series containing a range of data points over time for each time series
 - Scalar - a simple numeric floating point value
 - String - a simple string value

Instant Vector

- Containing the current value of that metric for each labeled instance at query time

```
<metric_name>{<label_selectors>}
```

- Examples:

```
node_cpu_seconds_total{mode="idle"}  
node_cpu_seconds_total{mode="idle",cpu="0"}
```

- Each vector may contain many values due to the use of labels as each one creates its own time series

Label Matching Operators

= Selects labels that are exactly equal to the provided string

!= Select labels that are not equal to the provided string

=~ Select labels that match a regular expression

!~ Select labels that do not match a regular expression

```
prometheus_http_requests_total{handler=~"/api/v1/.*"}  
prometheus_http_requests_total{code!="200"}
```

Range Vector

- Containing the current value of that metric for each labeled instance at query time

```
<metric_name>{<label_selectors>} [duration]
```

- Examples:

```
node_cpu_seconds_total{mode="idle",cpu="0"} [1h]  
prometheus_http_requests_total{code=~"4.."} [5m]
```

Expressions

- An expression can be a simple metric name (`node_cpu_seconds_total`) or a complex combination of functions, operators, and selectors
- Operator: PromQL provides arithmetic, comparison, logical, and aggregation operators for filtering and manipulating the metrics data (scalars, instant vectors, or combinations of both)
- Function: take instant vectors (or scalars and range vectors in some cases) as input, transform and analyze time series data, and return a new value, usually another instant vector

Expression Examples

- Calculating the percentage of memory used on a system

```
(1 - node_memory_MemFree_bytes / node_memory_MemTotal_bytes) * 100
```

`node_memory_MemTotal_bytes` = Total physical memory in bytes
(scalar)

`node_memory_MemFree_bytes` = Free memory in bytes (instant
vector)

Expression Examples

- Calculating the average cpu utilization of 1-minute range

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

`node_cpu_seconds_total` = Total CPU seconds in that specific mode

`node_cpu_seconds_total{mode="idle"}[1m]` = range vectors of 4 values each (config to measure every 15 seconds, 1 min = 4 values)

rate(v range-vector) calculates the per-second average rate of increase of the time series in the range vector v (return instant vector)

avg(v) divides the sum of v by the number of aggregated samples in the same way as the / binary operator

Aggregation Operators

- PromQL provides several aggregation operators for combining related time series (similar to groupby in pandas)

Operator	Description
sum	Calculates the sum of values across time-series.
avg	Computes the average of values across time-series.
min	Finds the minimum value across time-series.
max	Finds the maximum value across time-series.
count	Counts the number of time-series.
stddev	Calculates the standard deviation across time-series.
stdvar	Calculates the variance across time-series.
quantile	Computes a specific quantile (e.g., 0.95).
topk	Selects the top k time-series by value.
bottomk	Selects the bottom k time-series by value.

Aggregation Dimension Control

- Aggregation operator allows the use of “without” and “by” to exclude or include labels in the calculation

```
sum(node_cpu_seconds_total)
```

```
sum(node_cpu_seconds_total) without (cpu)
```

```
sum(node_cpu_seconds_total) by (cpu)
```

```
sum by (cpu) (node_cpu_seconds_total)
```

Using Python with Prometheus

- We can retrieve data from Prometheus server via Prometheus http api using prometheus_api_client python package. To use this package, we must install the following packages:

```
pip install prometheus_api_client  
pip install pandas
```

- prometheus_api_client provides several functions. See example in the jupyter notebook for more details

prometheus

prometheus.ipynb prometheus U X

prometheus.ipynb > M Retrieving Data from Prometheus > M We first import all necessary packages and connect to our local prometheus server

Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Select Kernel

9

Retrieving Data from Prometheus

We can retrieve data from Prometheus server via Prometheus http api using prometheus_api_client python package. To use this package, we must install the following packages:

```
pip install prometheus_api_client
pip install pandas
```

We first import all necessary packages and connect to our local prometheus server.

```
from prometheus_api_client import PrometheusConnect, MetricSnapshotDataFram
from prometheus_api_client.utils import parse_datetime
from datetime import timedelta
```

```
prom_url = "http://localhost:9090"
prom = PrometheusConnect(url=prom_url, disable_ssl=True)
```

Python

Python

< main* Launchpad 0 0 docker-desktop default Update is ready, click to restart. {} Cell 2 of 17

Conclusion

- Selecting proper datastore that suits to data characteristics is very important
- Time Series Database provides several important features for analyzing and handling time series data
- Prometheus is a very TSDB, especially for observability
- PromQL is Prometheus Query Language supporting data model with metrics and labels, built-in aggregation operators, and comprehensive time series functions

Recommended Resources

- <https://medium.com/kbtg-life/observability-pt-1-ทำความเข้าใจจกกับ-observability-กันเถอะ-5b2561bc4365>
- <https://wbassler23.medium.com/getting-started-with-prometheus-pt-1-8f95eef417ed>
- <https://betterstack.com/community/guides/monitoring/prometheus-metrics-explained/>
- <https://medium.com/@snehatuladhar098/beginners-hands-on-guide-monitor-your-computer-with-prometheus-and-grafana-e3c9011e47c8>
- <https://pypi.org/project/prometheus-api-client/>
- <https://promlabs.com/promql-cheat-sheet/>